

Traductor móvil en tiempo real para Lengua de Señas Argentina y voz, basado en aprendizaje automático con ejecución local

Integrante: Evelin Emilce Aragón

Rol: Desarrollo, entrenamiento de modelos, integración móvil y diseño arquitectónico.

1. Resumen descriptivo

Este proyecto propone el desarrollo de una aplicación móvil que permita la comunicación entre personas oyentes e hipoacúsicas mediante la traducción automática en tiempo real de la Lengua de Señas Argentina (LSA) a texto y voz, y de voz a texto. A diferencia de muchas soluciones existentes, el sistema funcionará completamente sin conexión a Internet, ejecutando todo el procesamiento de manera local en dispositivos Android.

La arquitectura se basa en dos dispositivos complementarios: uno dedicado a la captura de gestos mediante cámara y su interpretación como texto o voz; otro a la transcripción de la voz a texto visible. El presente trabajo propone construir un **prototipo funcional inicial** que permita validar la factibilidad de esta arquitectura, e incorporar un módulo experimental de aprendizaje federado para explorar estrategias de mejora colaborativa del modelo sin necesidad de centralizar los datos.

El enfoque del trabajo es técnico-aplicado, con fuerte componente experimental, orientado a validar la viabilidad del procesamiento local de señas en condiciones realistas de uso.

2. Fundamentación y estado del arte

El acceso equitativo a la comunicación sigue siendo una barrera crítica para personas hipoacúsicas en Argentina. Existen múltiples enfoques recientes en traducción automática de lengua de señas. El trabajo de Liang et al. [5] ofrece una taxonomía integral de las arquitecturas utilizadas en traducción automática de lengua de señas, distinguiendo entre enfoques basados en video crudo, glosas intermedias y coordenadas de pose. El presente proyecto se inscribe dentro de esta última categoría (pose-based), al emplear vectores de puntos clave como representación de entrada. Además, el estudio enfatiza la problemática de la escasez de datasets anotados, especialmente para lenguas señadas con baja representación internacional, lo cual refuerza la pertinencia de nuestra estrategia de construcción ad hoc de datos.

Algunos trabajos han explorado implementaciones completas con redes neuronales y visualización tridimensional mediante avatares [4], lo que refleja la diversidad de abordajes tecnológicos en el área. Sin embargo, la mayoría de las soluciones actuales presenta dos limitaciones recurrentes: requieren conexión constante a Internet y ejecutan el procesamiento en servidores externos, lo que compromete la autonomía, la privacidad y la accesibilidad real del sistema.

Por otro lado, tanto en el plano internacional como local, persiste una escasez significativa de datasets anotados para LSA. Este proyecto no escapa a esa dificultad, pero la aborda mediante la construcción propia de un conjunto de datos reducido, adaptado al entorno experimental [7]. Esta estrategia permite mantener un control preciso sobre las condiciones de captura, usuarios, y tipo de vocabulario, lo cual es fundamental para validar modelos con inferencia local.

En cuanto a la arquitectura técnica, se propone un modelo liviano basado en *landmarks*, entrenado con redes LSTM y optimizado con TensorFlow Lite para correr directamente en Android. A diferencia de sistemas centralizados, aquí toda la inferencia ocurre en el dispositivo, sin transferencia de datos ni infraestructura adicional.

Como componente exploratorio, se incorpora un módulo experimental de *federated learning*, inspirado en producciones como GBoard [15], donde se entrena un modelo de predicción de texto directamente en el dispositivo del usuario. Este paradigma de entrenamiento descentralizado también ha sido aplicado en contextos médicos para preservar la privacidad de datos sensibles [10], y en vehículos inteligentes [11]. Estas investigaciones no solo demuestran la aplicabilidad del enfoque federado a modelos de voz, sino que también abordan distintos aspectos críticos del problema: desde el modelado acústico en escenarios no IID (datos no independientes ni

distribuidos de forma idéntica) [12], hasta la medición de la pérdida de calidad frente a centralización [13], y la protección de la identidad del usuario mediante privacidad diferencial [14].

En este sentido, el algoritmo *Federated Averaging (FedAvg)* propuesto por McMahan et al. [6] representa un avance clave. Este enfoque permite combinar entrenamientos locales realizados en cada dispositivo mediante un promedio ponderado de los modelos resultantes, reduciendo considerablemente la cantidad de rondas de comunicación necesarias para alcanzar un rendimiento satisfactorio. Los autores demostraron que FedAvg logra eficiencias de 10x a 100x en escenarios reales de entrenamiento distribuido, incluso en presencia de datos no balanceados y distribuciones no IID, condiciones que se alinean con el caso abordado en este trabajo.

3. Arquitectura general del sistema propuesto

El sistema planteado se compone de dos dispositivos móviles Android que operan de forma complementaria y descentralizada. Uno de ellos está destinado a la captura y traducción de señas, mientras que el otro ejecuta la transcripción de voz a texto. Ambos dispositivos funcionan de manera completamente local, sin requerir conexión entre sí ni acceso a Internet.

La arquitectura incluye los siguientes módulos principales:

- **Captura de imagen:** accede a la cámara del dispositivo para registrar gestos en tiempo real.
- **Extracción de *landmarks*:** utiliza MediaPipe [1], una biblioteca desarrollada por Google que permite detectar en tiempo real puntos clave tridimensionales de manos, rostro y cuerpo, mediante una arquitectura optimizada basada en convoluciones separables. Este enfoque ha demostrado ser altamente eficiente para dispositivos móviles con recursos limitados.
- **Preprocesamiento:** organiza los datos en secuencias temporales normalizadas, asegurando la coherencia en la entrada del modelo secuencial.
- **Modelo de clasificación LSTM:** se emplea una red neuronal recurrente tipo LSTM (Long Short-Term Memory), ampliamente utilizada en tareas de secuencias por su capacidad para capturar dependencias temporales a largo plazo. Esta arquitectura ha sido validada en estudios de clasificación de movimientos basados en secuencias de pose.
- **Conversión a texto y voz:** transforma la predicción en subtítulos en pantalla y en audio mediante el módulo *TextToSpeech* nativo de Android. Esta herramienta permite sintetizar voz de forma local, sin necesidad de conexión externa.
- **Entrenamiento federado:** permite el refinamiento distribuido del modelo de clasificación, sincronizando sus actualizaciones sin transferir datos. Su integración sigue los lineamientos planteados por McMahan et al. para el algoritmo FedAvg [6], asegurando eficiencia en entornos no IID.

A continuación se presenta un esquema de la arquitectura funcional del sistema en modo *LSA-texto/voz*:

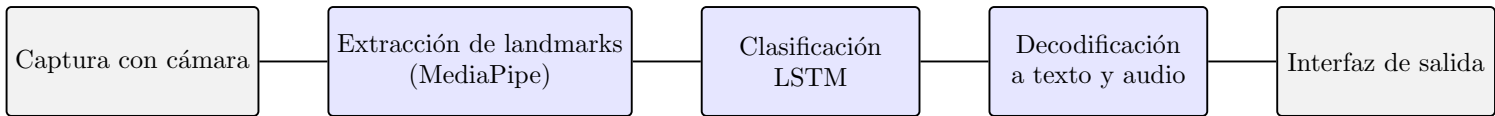


Figura 1: Flujo funcional del sistema - Modo LSA a texto/voz

Este diagrama sintetiza el flujo principal en el dispositivo de reconocimiento de señas. El segundo dispositivo opera en forma independiente, capturando voz mediante micrófono y transcribiendo a texto mediante herramientas de reconocimiento de voz del sistema operativo Android.

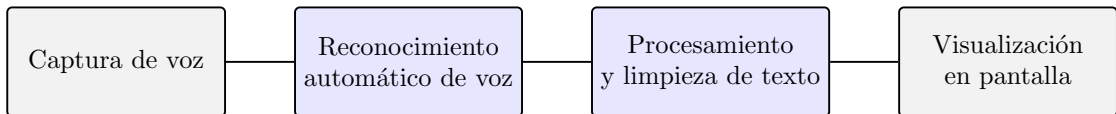


Figura 2: Flujo funcional del sistema - Modo voz a texto

3.1. Representación de las señas

Cada seña se modelará como una secuencia temporal de T cuadros consecutivos. En cada uno de ellos se capturarán las posiciones tridimensionales (x, y, z) de $N = 21 \times 2 = 42$ puntos clave correspondientes a ambas manos, extraídos mediante MediaPipe. Esto da lugar a un vector $\vec{x}_t \in \mathbb{R}^{126}$ por cuadro, compuesto por la concatenación de los 42 tripletes.

La seña completa queda entonces representada por una matriz:

$$\mathbf{X} \in \mathbb{R}^{T \times 126}$$

donde cada fila $\vec{x}_t$ representa un instante temporal, y las columnas codifican la configuración gestual de las manos. Esta representación preserva la dinámica temporal y permite su procesamiento mediante redes neuronales recurrentes como LSTM, que capturan las dependencias entre cuadros sucesivos.

Se adoptará una longitud fija $T = 15$, normalizando la duración de todas las muestras por interpolación o truncamiento. Esto permite que el modelo procese entradas homogéneas sin necesidad de mecanismos adicionales de padding o segmentación.

3.2. Modelo propuesto

El modelo de clasificación utilizado es una red neuronal profunda con arquitectura secuencial LSTM, diseñada para capturar las dependencias temporales entre cuadros consecutivos de una seña. La entrada es una matriz $\mathbf{X} \in \mathbb{R}^{15 \times 126}$, donde cada fila representa los 126 valores (x, y, z) de los 42 puntos clave capturados por frame.

La arquitectura propuesta es la siguiente:

- LSTM(64, `return_sequences=True`) seguido de Dropout(0.5).
- LSTM(128, `return_sequences=False`) seguido de Dropout(0.5).
- Tres capas densas: Dense(64, ReLU), Dense(64, ReLU), Dense(C, softmax).

El primer bloque LSTM procesa cada secuencia de forma temporal, emitiendo una salida por cuadro. Esta salida es filtrada por dropout para evitar sobreajuste. Luego, la segunda LSTM resume la secuencia en un único vector de 128 dimensiones, que constituye una representación latente de la seña completa. Este vector pasa por un pipeline denso no lineal que actúa como decodificador hacia la distribución de clases.

Se optó por una doble capa LSTM para permitir una codificación jerárquica: la primera LSTM actúa como extractor local de patrones, mientras que la segunda integra esta información en un resumen global. Las capas densas finales permiten una transformación no lineal de este resumen hacia el espacio de clases posibles.

Esta arquitectura logra un compromiso entre expresividad y eficiencia computacional, manteniendo una huella liviana para su posterior inferencia en dispositivos móviles vía TensorFlow Lite.

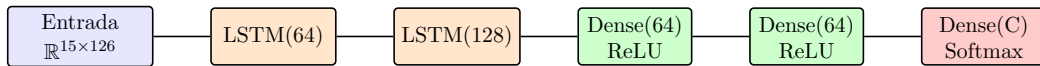


Figura 3: Modelo LSTM propuesto para la clasificación de secuencias de señas

Módulo experimental de aprendizaje federado

Como componente exploratorio, se incorpora un módulo de *aprendizaje federado* [3] como estrategia colaborativa para mejorar el modelo de reconocimiento de señas sin centralizar datos. Este enfoque se justifica particularmente en escenarios móviles con conectividad limitada, donde la eficiencia de comunicación es crítica. Konečný et al. proponen dos técnicas para abordar este problema: (i) actualizaciones estructuradas, que imponen restricciones como bajo rango o esparsidad a los gradientes locales para reducir su tamaño; y (ii) actualizaciones esbozadas, donde se aplican transformaciones como rotaciones aleatorias y cuantización antes de enviar los parámetros. Estas técnicas permiten entrenar modelos complejos con un costo de comunicación hasta 100× menor, manteniendo un desempeño competitivo, lo que las hace especialmente relevantes para una arquitectura distribuida basada en Android.

En ausencia de grandes datasets públicos de LSA, el aprendizaje federado propone entrenar modelos de manera descentralizada en dispositivos de usuarios, evitando compartir los datos crudos [8].

En este trabajo se adopta un esquema de *FedAvg* (Federated Averaging) [6], configurado para dispositivos Android con conjuntos de datos reducidos y variables. La implementación contempla condiciones de entrenamiento asincrónico y heterogéneo, donde cada usuario puede tener distintos patrones gestuales y capacidades computacionales. Esta adaptación busca validar empíricamente la viabilidad de aplicar aprendizaje federado en contextos reales de accesibilidad, priorizando eficiencia comunicacional sin comprometer la privacidad.

El uso de frameworks como **TensorFlow Federated** o **Flower** permitirá ensayar esta estrategia en Android, evaluando su factibilidad en términos de consumo de recursos y ganancia en precisión. Un estudio comparativo exhaustivo llevado a cabo por Wang et al. [9] evaluó el rendimiento de distintos frameworks de federated learning en dispositivos reales, concluyendo que tanto TensorFlow Federated como Flower ofrecen una buena combinación de escalabilidad, eficiencia y compatibilidad con entornos móviles, lo que justifica su selección en este proyecto.

Cabe destacar que no se prevé realizar un benchmarking comparativo con otras aplicaciones existentes, ya que la mayoría de los sistemas comerciales disponibles requieren conexión a internet, no publican sus modelos ni códigos fuente, y operan bajo arquitecturas distintas (centralizadas y basadas en la nube). Esta diferencia de condiciones hace inviable una comparación justa o replicable. El foco de este trabajo radica en la validación empírica del rendimiento en entorno local, bajo restricciones reales de ejecución en dispositivos móviles.

3.3. Alcances y limitaciones del análisis comparativo

A diferencia de trabajos previos sobre traducción automática de lengua de señas o reconocimiento gestual, este proyecto no contempla un análisis comparativo directo con aplicaciones comerciales existentes. Esta decisión se fundamenta en la falta de condiciones replicables: la mayoría de los sistemas actuales son cerrados, no publican sus modelos ni datasets, y requieren conexión permanente a servidores remotos. Además, sus arquitecturas están optimizadas para procesamiento centralizado en la nube, lo cual contradice el enfoque distribuido y local propuesto aquí.

En su lugar, se adopta una estrategia de validación empírica focalizada, midiendo el rendimiento del sistema en condiciones reales de ejecución en dispositivos móviles. Esto incluye pruebas de inferencia local, sensibilidad a variaciones gestuales y evaluación de robustez estadística frente a perturbaciones menores en la entrada. Este tipo de validación ha sido defendido en trabajos recientes sobre estabilidad de modelos [16, 17, 18] como un criterio esencial en tareas donde la coherencia de las predicciones es tan relevante como su precisión nominal.

El foco del análisis reside, por tanto, en determinar la viabilidad técnica y la consistencia funcional del sistema propuesto en escenarios de accesibilidad concreta, sin depender de benchmarks artificiales o condiciones idealizadas.

3.4. Entrenamiento e inferencia en entorno móvil

El modelo se entrenará utilizando **Keras** con backend **TensorFlow**, sobre secuencias de puntos clave extraídas con MediaPipe. El conjunto de datos será construido ad hoc mediante capturas de gestos en condiciones controladas. Una vez alcanzado un rendimiento satisfactorio en validación cruzada, se procederá a la conversión del modelo al formato **TensorFlow Lite (TFLite)**, optimizado para dispositivos móviles [2].

Para garantizar eficiencia de ejecución y bajo consumo de recursos, se evaluarán técnicas de *cuantización post-entrenamiento*, tales como:

- **Cuantización dinámica:** reduce el tamaño del modelo y acelera inferencias sin requerir datos de calibración.
- **Cuantización entera:** convierte todos los pesos y activaciones a enteros, usando un subconjunto del set de entrenamiento como calibración.

El modelo TFLite resultante se integrará en una aplicación Android desarrollada en **Kotlin**, utilizando la API **Interpreter** provista por TensorFlow. Se prevé la carga del modelo como recurso embebido, sin dependencias externas ni conexión a internet. Esta configuración permite inferencia local en tiempo real sobre frames capturados por la cámara del dispositivo, alineándose con los principios de eficiencia y autonomía de TFLite para entornos de borde [2].

Este enfoque asegura portabilidad, autonomía y bajo requerimiento computacional, alineado con el objetivo de accesibilidad técnica para usuarios finales. Además, la arquitectura modular permite actualizar el modelo de forma incremental, incorporando mejoras futuras sin alterar la lógica de captura ni la interfaz de usuario.

3.5. Evaluación y robustez del modelo

Durante el entrenamiento supervisado, se emplearán como métricas principales la **precisión** (*accuracy*), que indica el porcentaje total de predicciones correctas, y la **pérdida categórica cruzada** (*categorical crossentropy*), que cuantifica el grado de error entre la distribución de probabilidad esperada y la predicha.

Para análisis cualitativo del desempeño por clase, se utilizarán también las métricas **F1-score macro** y **F1-score micro**. El F1 macro da igual peso a cada clase, independientemente de su frecuencia, y resulta útil para evaluar el rendimiento global cuando las clases están desbalanceadas. El F1 micro, en cambio, pondera según la frecuencia real de aparición, reflejando mejor el comportamiento general del modelo sobre el conjunto de datos completo. Estas métricas se complementan con matrices de confusión para un diagnóstico detallado de errores específicos.

Para evaluar el comportamiento del modelo en condiciones no ideales, se testeará su robustez ante variaciones de:

- Usuario (diferentes morfologías de mano, estilos de seña y velocidad de ejecución).
- Iluminación (condiciones interiores, exteriores, sombras).
- Ángulo y distancia de la cámara.

Dado que el sistema se usará en tiempo real y en entornos sin etiquetas, se propone una métrica adicional de **consistencia estadística**, basada en la hipótesis de que entradas similares deberían inducir salidas similares. Se define:

$$\sigma = \text{std}(\hat{y}_i) \quad \text{para todas } i \text{ tales que } \|\mathbf{X}_i - \mathbf{X}_j\| < \varepsilon$$

donde $\hat{y}_i$ es la salida del modelo (probabilidades o clase predicha), $\mathbf{X}_i$ es la secuencia de entrada correspondiente, y ε representa un umbral de vecindad en el espacio de entrada: define cuán “cerca” deben estar dos gestos para ser considerados similares. Un valor bajo de σ indica que el modelo es estable frente a variaciones sutiles en los datos de entrada, lo cual es deseable en producción.

Consistencia estadística: definición computacional

Para operacionalizar la métrica de consistencia estadística propuesta, se implementa el siguiente procedimiento basado en cálculos sobre la vecindad local de cada entrada:

1. Se parte del conjunto de entradas $\{\mathbf{X}_i\}_{i=1}^n$ y sus respectivas salidas $\{\hat{y}_i\}_{i=1}^n$, donde:
 - n : cantidad total de muestras en el conjunto.
 - $\mathbf{X}_i$: secuencia de entrada de la muestra i .
 - $\hat{y}_i \in \mathbb{R}^C$: vector de probabilidades predicho por el modelo para C clases.
2. Se define un umbral $\varepsilon > 0$ que determina la vecindad en el espacio de entrada: dos entradas $\mathbf{X}_i$ y $\mathbf{X}_j$ son vecinas si $\|\mathbf{X}_i - \mathbf{X}_j\| < \varepsilon$.
3. Para cada entrada $\mathbf{X}_i$, se construye el conjunto de vecinos:

$$\mathcal{N}_i = \{j \mid \|\mathbf{X}_i - \mathbf{X}_j\| < \varepsilon\}$$

4. Se computa la desviación estándar local de las predicciones sobre los vecinos:

$$\sigma_i = \text{mean}_c \left(\text{std}_{\text{sesgada}} (\{\hat{y}_{jc} \mid j \in \mathcal{N}_i\}) \right),$$

donde:

- $\hat{y}_{jc}$: probabilidad asignada a la clase c por la predicción $\hat{y}_j$.

- $\text{std}_{\text{sesgada}}$: desviación estándar de las probabilidades predichas para una clase fija c , evaluada sobre los vecinos $j \in \mathcal{N}_i$ de la muestra i . Se calcula como:

$$\text{std}_{\text{sesgada}}(\hat{y}_{1c}, \dots, \hat{y}_{mc}) = \sqrt{\frac{1}{m} \sum_{j=1}^m (\hat{y}_{jc} - \bar{y}_c)^2}, \quad \text{con } \bar{y}_c = \frac{1}{m} \sum_{j=1}^m \hat{y}_{jc}$$

donde $\hat{y}_{jc}$ representa la probabilidad asignada a la clase c por la salida del modelo sobre el gesto j , y $m = |\mathcal{N}_i|$ es la cantidad de vecinos de $\mathbf{X}_i$. Esta medida cuantifica cuán estables son las probabilidades del modelo frente a pequeñas perturbaciones en la entrada (vecindad gestual).

- mean_c : promedio sobre todas las clases c .

5. Finalmente, la métrica global es el promedio sobre todos los i con vecindad no vacía:

$$\sigma = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \sigma_i, \quad \text{donde } \mathcal{I} = \{i \mid |\mathcal{N}_i| > 1\}$$

Notación utilizada:

- n : cantidad total de muestras en el conjunto. Cada muestra representa una secuencia capturada de una señal en LSA.
- $\mathbf{X}_i$: entrada de la muestra i , es decir, una secuencia temporal de vectores de coordenadas de puntos clave (por ejemplo, posiciones de manos durante la señal).
- $\hat{y}_i \in \mathbb{R}^C$: vector de probabilidades predicho por el modelo para las C clases de señales. Por ejemplo, $\hat{y}_i = [0,05, 0,02, 0,88, 0,05]$ indica alta probabilidad para la clase 3.
- $\hat{y}_{jc}$: componente escalar del vector $\hat{y}_j$, representa la probabilidad que el modelo asigna a la clase c para la entrada $\mathbf{X}_j$. Es un valor entre 0 y 1.
- $\mathcal{N}_i$: conjunto de *vecinos* de $\mathbf{X}_i$, es decir, otras entradas $\mathbf{X}_j$ tales que la distancia euclídea entre $\mathbf{X}_i$ y $\mathbf{X}_j$ es menor a un umbral ε .
- $\{j \mid \dots\}$: notación por comprensión; contiene todos los índices j que satisfacen la condición dada.
- $|\mathcal{N}_i|$: número de vecinos que tiene la entrada $\mathbf{X}_i$.
- σ_i : valor escalar que resume la variabilidad de las predicciones para los vecinos de $\mathbf{X}_i$. Si σ_i es baja, el modelo predice de forma consistente en esa zona del espacio de gestos.
- $\mathcal{I}$: subconjunto de índices i tales que $\mathbf{X}_i$ tiene al menos dos vecinos (para que tenga sentido calcular la desviación estándar).
- $|\mathcal{I}|$: número de muestras con vecinos suficientes para ser consideradas en la métrica final.

Interpretación complementaria:

- Considérese que una misma señal pueda ejecutarse múltiples veces, con pequeñas variaciones en la posición, velocidad o forma de las manos.
- Un modelo robusto debería generar predicciones similares para todas estas ejecuciones, ya que corresponden a la misma clase gestual. Es decir, sus salidas deberían coincidir en la clase asignada y tener distribuciones de probabilidad parecidas.
- Lo que hace esta métrica es, para cada muestra del conjunto, identificar cuáles otras están suficientemente “cerca” en el espacio de entrada, lo que implica que representan gestos similares en su forma.
- Luego, compara las predicciones del modelo para esa muestra y sus vecinas. Si todas esas salidas son consistentes entre sí, se asume que el modelo es estable en esa región del espacio gestual.
- En cambio, si pequeñas diferencias en la entrada provocan respuestas muy distintas (por ejemplo, asignar clases distintas o con probabilidades muy dispares), eso se refleja en una alta desviación estándar, y se interpreta como falta de consistencia local.

Fundamentación del enfoque:

El enfoque propuesto se inspira en principios teóricos de *continuidad* y *estabilidad* de funciones predictoras: si dos entradas $\mathbf{X}_i$ y $\mathbf{X}_j$ son cercanas en el espacio de entrada (por ejemplo, dos videos que representan el mismo gesto con pequeñas variaciones de iluminación, posición o velocidad), se espera que el modelo produzca salidas

similares. Esta propiedad es deseable en sistemas de reconocimiento de señas en contexto real, donde el ruido y la variabilidad son inevitables.

El uso de la desviación estándar de las predicciones en vecindades locales permite cuantificar empíricamente cuán sensible es el modelo a pequeñas perturbaciones de entrada. Hein et al. [17] formalizan esta noción de estabilidad en redes neuronales al estudiar la variabilidad de las salidas frente a perturbaciones controladas, proponiendo medidas teóricas de sensibilidad para arquitecturas ReLU. Por su parte, Jiang et al. [16] introducen el *Trust Score* como métrica de coherencia local, al comparar la asignación de clase del modelo con la distancia a regiones de alta densidad en el espacio de representación. Más recientemente, Manav et al. [18] evalúan la *consistencia semántica* en modelos de clasificación visual, sugiriendo que la robustez debe medirse también en contextos de variabilidad leve pero significativa.

En el contexto de este proyecto, la métrica de consistencia estadística permite complementar las evaluaciones clásicas de *accuracy* y *F1-score*, que no capturan inestabilidades locales. Un modelo con alta precisión pero que responde erráticamente a señas similares compromete la fiabilidad del sistema en uso real. Por tanto, esta estrategia permite evaluar de forma directa cuán *coherente* es el modelo ante datos del mismo concepto gestual que podrían presentarse en condiciones variables.

4. Riesgos metodológicos y estrategias de mitigación

La propuesta contempla distintos factores de riesgo que podrían afectar la validez, funcionalidad o escalabilidad del sistema. A continuación se identifican los principales riesgos metodológicos previstos, junto con las estrategias diseñadas para mitigarlos.

Categoría	Riesgo identificado	Estrategia de mitigación
Calidad del dataset	La falta de un corpus anotado localmente obliga a recolectar y etiquetar datos propios, lo que puede introducir sesgos o limitaciones de volumen.	Diseñar un esquema controlado de captura de datos con diversidad de usuarios, condiciones de iluminación y posturas. Priorizar un etiquetado manual riguroso.
Overfitting del modelo	Dada la escasez de datos, existe riesgo de sobreajuste a las condiciones del entrenamiento inicial.	Aplicar técnicas de regularización (Dropout), validación cruzada y testeo en datos no vistos. Analizar desempeño por usuario.
Generalización interusuario	El modelo podría tener dificultad para reconocer señas de personas distintas a las del conjunto de entrenamiento.	Incentivar la recolección distribuida de gestos, contemplando múltiples perfiles físicos y estilos de seña.
Instabilidad del aprendizaje federado	El entrenamiento descentralizado puede amplificar errores si los clientes refuerzan patrones erróneos sin control.	Establecer métricas de consistencia local, limitar la magnitud de actualizaciones y validar manualmente los casos atípicos en fases de prueba.
Performance en dispositivos móviles	Las redes LSTM pueden implicar tiempos de inferencia elevados, afectando la experiencia en tiempo real.	Cuantizar el modelo con TFLite, analizar tiempos de ejecución y, si es necesario, simplificar arquitectura sin comprometer precisión.

5. Plan de trabajo (síntesis)

El desarrollo se organizará en seis fases consecutivas, que permitirán abordar progresivamente los componentes técnicos y validar funcionalmente la propuesta. El siguiente cronograma resume las actividades principales por mes:

Fase	Actividad principal
Fase 1 (Mayo - Junio)	Implementación de pipeline de captura de gestos. Extracción de landmarks con MediaPipe. Diseño preliminar de arquitectura del modelo y pruebas de inferencia básica.
Fase 2 (Julio)	Entrenamiento supervisado del modelo LSTM en entorno local. Evaluación con conjunto reducido de señas. Optimización y conversión a TensorFlow Lite.
Fase 3 (Agosto)	Desarrollo de la interfaz dual en Android con integración del modelo. Validación funcional en entorno móvil. Recolección inicial de métricas de desempeño.
Fase 4 (Septiembre)	Diseño e integración de módulo experimental de aprendizaje federado. Simulación de entrenamiento distribuido en instancias controladas.
Fase 5 (Octubre)	Validación sistemática: análisis de precisión, F1-score y consistencia. Pruebas de robustez ante condiciones variables. Refinamiento del modelo.
Fase 6 (Noviembre)	Elaboración del informe final, documentación técnica, anexos y entrega de demo funcional. Preparación para defensa oral.

Referencias

- [1] Lugaresi, C., et al. (2019). *MediaPipe: A Framework for Building Perception Pipelines*. arXiv:1906.08172. Disponible en: <https://arxiv.org/abs/1906.08172>
- [2] TensorFlow. (2023). *TensorFlow Lite Guide*. Disponible en: <https://www.tensorflow.org/lite/>
- [3] Konečný, J., et al. (2017). *Federated Learning: Strategies for Improving Communication Efficiency*. arXiv:1610.05492. Disponible en: <https://arxiv.org/abs/1610.05492>
- [4] Shovkovyi, V., et al. (2023). *Automatic Sign Language Translation System Using Neural Network Technologies and 3D Animation*. IEEE. Disponible en: https://www.researchgate.net/publication/376880024_Automatic_sign_language_translation_system_using_neural_network_technologies_and_3D_animation
- [5] Liang, R., et al. (2023). *Sign Language Translation: A Survey*. Electronics, 12(12), 2678. Disponible en: <https://www.mdpi.com/2079-9292/12/12/2678>
- [6] McMahan, B., et al. (2017). *Communication-efficient learning of deep networks from decentralized data*. AISTATS. arXiv:1602.05629. Disponible en: <https://arxiv.org/abs/1602.05629>
- [7] Koller, O., et al. (2022). *Challenges with Sign Language Datasets*. Proceedings of the LREC 2022. Disponible en: <https://aclanthology.org/2022.lrec-1.264/>
- [8] Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2019). Federated Learning: Challenges, Methods, and Future Directions. arXiv preprint arXiv:1908.07873. Disponible en: <https://arxiv.org/abs/1908.07873>
- [9] Wang, J., Charles, Z., Xu, Z., Joshi, G., McMahan, H. B., Aguera y Arcas, B., ... & Zhu, C. (2021). A Field Guide to Federated Optimization. arXiv preprint arXiv:2107.06917. Disponible en: <https://arxiv.org/abs/2107.06917>
- [10] Sheller MJ, Edwards B, et al. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Sci Rep*. 2020;10:12598. <https://www.nature.com/articles/s41598-020-69250-1>
- [11] Elbir, A. M., Soner, B., Coleri, S., Gündüz, D., & Bennis, M. Federated Learning for Vehicular Networks. *2022 IEEE International Mediterranean Conference on Communications and Networking (MeditCom)*. https://www.researchgate.net/publication/365100751_Federated_Learning_in_Vehicular_Networks
- [12] Cui, X., Lu, S., Kingsbury, B. (2021). *Federated Acoustic Modeling For Automatic Speech Recognition*. arXiv:2102.04429. <https://arxiv.org/abs/2102.04429>
- [13] Guliani, D., Beaufays, F., Motta, G. (2020). *Training Speech Recognition Models with Federated Learning: A Quality/Cost Framework*. arXiv:2010.15965. <https://arxiv.org/abs/2010.15965>
- [14] Granqvist, F., Seigel, M., van Dalen, R., Cahill, Á., Shum, S., Paulik, M. (2020). *Improving on-device speaker verification using federated learning with privacy*. Interspeech 2020. <https://arxiv.org/abs/2008.02651>
- [15] Hard, A., et al. (2018). *Federated Learning for Mobile Keyboard Prediction*. arXiv:1811.03604. Disponible en: <https://arxiv.org/abs/1811.03604>
- [16] Jiang, Z., He, H., Shah, D., Thiagarajan, J. J., & Hodas, N. (2021). *How Can We Know What Language Models Know?* <https://arxiv.org/abs/1911.12543>
- [17] Hein, M., & Andriushchenko, M. (2019). *Why ReLU networks yield high-confidence predictions far away from the training data and how to mitigate the problem*. <https://arxiv.org/abs/1812.05720>
- [18] Manav Chaudhary, Harshit Gupta, Savita Bhat, Vasudeva Varma (2024). *Towards Understanding the Robustness of LLM-based Evaluations under Perturbations*. <https://aclanthology.org/2024.icon-1.22.pdf>